

Bot AI System Guide

Spaceship Racing Project

Assets/Scripts overview

May 5, 2026

Contents

1	What This Document Covers	3
2	Quick Unity Setup	3
3	Overall Architecture	3
4	AI Files	4
4.1	BotAIType.cs	4
4.2	Bot.cs	5
4.3	BotAgent.cs	5
4.4	BotAgentSensors.cs	5
4.5	BotAgentTargeting.cs	6
4.6	BotAgentRewards.cs	6
4.7	BotAgentCollision.cs	7
4.8	BotCheckpointTrigger.cs	8
4.9	BotRewardSystem.cs	8
4.10	FSMBotBrain.cs	8
4.11	RLBotBrain.cs	8
4.12	BotManager.cs	9
4.13	BotDebugUI.cs	9
5	Non-AI Scripts Used By Bots	10
5.1	Checkpoint.cs	10
5.2	FinishTrigger.cs	10
5.3	RaceManager.cs	10
5.4	PauseMenu.cs	10
5.5	FollowCamera.cs	11
5.6	Coin.cs	11

5.7 Bomb.cs	11
6 How Checkpoints Work For Bots	12
7 How Coins And Bombs Work For Bots	12
8 How Recovery Works	12
9 Debugging And Gizmos	13
10 Tuning Notes	13
11 What Is Still Basic	14

1 What This Document Covers

This document explains the bot AI code currently living in `Assets/Scripts`, with most focus on `Assets/Scripts/AI`. It is written for the current Stage 3 goal: basic bots that can race through checkpoints, collect coins, avoid bombs, and expose enough debug data to tune them in Unity.

The project already has the player, coins, bombs, checkpoints, and finish line. The bot layer adds a parallel racer that uses the same level objects but tracks its own checkpoint progress.

2 Quick Unity Setup

To run the bot system in Unity, the scene needs these references wired:

- Add a `BotManager` in the scene.
- Assign `BotManager.botPrefab` to the bot prefab.
- Assign `BotManager.botSpawnPoint`; usually this is the same start transform used by the player.
- Assign `BotManager.cameraRig` to the object that has `FollowCamera`, or leave it to auto-find if the scene only has one.
- On pause menu buttons, call wrapper methods on `PauseMenu`.
- Bot buttons can call `AddBot`, `RemoveBot`, `SwitchViewToBot`, and `ToggleBotAIGizmos`.
- If using `BotDebugUI`, assign a `TextMeshPro` text object to `debugText`. Assigning `botManager` is optional because it can auto-find it.
- Coins should have the `Coin` script and ideally the `Coin` tag.
- Bombs should have the `Bomb` script and ideally the `Bomb` tag.
- Checkpoints should have `Checkpoint`.
- Optional bot-only trigger objects can use `BotCheckpointTrigger`.
- If using those bot-only triggers, set explicit checkpoint indexes on them.

The bot prefab should have these components:

- `Bot`: movement controller.
- `BotAgent`: AI coordinator.
- `BotRewardSystem`: reward tracking for RL and debug.
- `FSMBotBrain`: main Stage 3 brain.
- `RLBotBrain`: basic RL skeleton, if the prefab can switch to RL.
- A collider somewhere on the bot, so overlap and trigger checks have bounds.

3 Overall Architecture

The AI is split into small files using a partial `BotAgent` class. The important idea is that `BotAgent` owns the race context, while the brain only decides steering and speed.

Part

Job

<code>Bot.cs</code>	Moves the ship forward, rotates it, handles acceleration/braking, and displays the nameplate. It does not decide where to go.
<code>BotAgent*.cs</code>	Finds checkpoints, coins, bombs, tracks progress, decides the current state, handles checkpoint/coin/bomb interactions, and gives target positions to the brain.
<code>FSMBotBrain.cs</code>	Simple hand-made AI brain. Steers toward the target and picks a safe speed depending on angle, distance, bombs, and recovery state.
<code>RLBotBrain.cs</code>	Basic Q-table demo. It still steers mostly toward the target, but can bias actions and update values from reward changes.
<code>BotRewardSystem.cs</code>	Stores reward numbers used by debug and RL.
<code>BotManager.cs</code>	Spawns/removes bots, tracks the bot list, switches camera view, and toggles AI gizmos.
<code>BotDebugUI.cs</code>	Displays live bot info on the screen.

One frame of bot AI looks like this:

1. `BotAgent.Update` checks that checkpoints exist.
2. It adds a small time penalty.
3. It scans for nearest checkpoint, useful coin, and dangerous bomb.
4. It checks overlap against coins, bombs, and the current checkpoint.
5. It updates the current state: `Racing`, `Collect`, `AvoidDanger`, or `Recover`.
6. It applies progress reward.
7. It ticks either the FSM brain or the RL brain.
8. The selected brain calls `Bot.SetAiInput`.
9. `Bot.Update` turns, accelerates/brakes, and moves forward.

4 AI Files

4.1 `BotAiType.cs`

Defines the small enums used by the AI:

- `BotAiType.FiniteStateMachine`: use the normal FSM brain.
- `BotAiType.ReinforcementLearning`: use the RL brain.
- `BotState.Racing`: normal checkpoint chasing.
- `BotState.AvoidDanger`: a bomb is close enough and in the flight corridor.
- `BotState.Collect`: a coin is worth steering toward.
- `BotState.Recover`: the bot missed a checkpoint or passed its plane badly and must line up again.

4.2 Bot.cs

Bot is the physical movement script for a bot ship.

Main responsibilities:

- Reads AI inputs set by `SetAiInput`.
- Smooths pitch and yaw input.
- Rotates the ship using pitch/yaw speeds.
- Adds visual roll while turning.
- Accelerates or brakes toward whatever the brain requested.
- Moves forward every frame using `transform.forward * currentSpeed`.
- Rotates the nameplate so it faces the camera.

Important point: this script is not pathfinding. It is closer to a bot version of the player movement controller. The brain says “turn left, pitch up, brake”, and Bot performs it.

4.3 BotAgent.cs

BotAgent is the center of the AI system. It is a partial class, so its logic is split across several files.

This file contains:

- Main settings for checkpoint paths, sensors, recovery, coins, bombs, and runtime state.
- References to Bot, BotRewardSystem, FSMBotBrain, and RLBotBrain.
- The core `Update` loop.
- Auto-finding checkpoints.
- Brain selection between FSM and RL.
- State selection.
- Gizmo drawing.

Checkpoint discovery works in two passes:

- First it looks for `BotCheckpointTrigger` objects and sorts by their explicit `checkpointIndex`.
- If those are not present, it falls back to `Checkpoint` objects sorted by hierarchy order.

The state logic is intentionally simple. Bomb danger wins first, recovery wins next, coin collection wins after that, and normal racing is the fallback.

4.4 BotAgentSensors.cs

This file scans the scene for things the bot cares about.

`FindNearestCheckpoint` is mostly for debug display. The actual race target is `currentTarget`, not just whatever checkpoint is nearest.

`FindDangerBombInFront` finds bombs that are:

- tagged Bomb,

- inside detection distance,
- in front of the bot,
- close enough sideways/vertically to matter,
- not too far off-angle.

This is why bomb avoidance does not trigger from every bomb in the level. It only reacts to bombs inside the forward corridor.

`FindBestPathCoin` finds coins that are:

- tagged `Coin`,
- in front of the bot,
- inside detection radius,
- not too close,
- reasonably aligned with the current checkpoint path.

Coins are treated as a small steering bias, not as the main objective. The checkpoint route still matters more.

4.5 `BotAgentTargeting.cs`

This file answers the question: “where should the bot point right now?”

Main methods:

- `GetTargetPositionByState`: final target position after coin bias and bomb avoidance.
- `GetCurrentTargetPosition`: center point of the current checkpoint.
- `GetCheckpointAimPosition`: aims at the checkpoint center, preferring the `portalSurface` bounds when available.
- `GetCheckpointRecoveryApproachPoint`.
- Creates a point before the checkpoint.
- That recovery point lets the bot go away, turn around, and line up.
- `ApplyCoinBias`: gently pulls target position toward a good coin.
- `ApplyBombAvoidance`: pushes target position away from dangerous bombs.

Recovery is the important part for missed checkpoints. If the bot has flown past the checkpoint plane or is too badly angled, it does not keep circling the checkpoint. It picks a setup point before the checkpoint, flies there, then turns back through the checkpoint.

4.6 `BotAgentRewards.cs`

This file tracks progress and checkpoint completion.

Main responsibilities:

- Adds reward for moving closer to the current target.
- Adds small reward for moving fast while aligned.

- Penalizes wrong direction progress.
- Detects checkpoint overlap using bounds, not only Unity trigger events.
- Completes checkpoints only in the correct order.
- Verifies the bot is facing through the checkpoint using a dot product threshold.
- Tracks `passedCheckpointCount`.
- Exposes `HasPassedAllCheckpoints` for the finish line.

The checkpoint counter is independent from the player counter in `RaceManager`. The bot has its own progress count because checkpoints should not disappear or become disabled for every racer.

4.7 `BotAgentCollision.cs`

This file handles collision and trigger events for bot interactions.

It handles:

- Checkpoints.
- Coins.
- Bombs.
- Respawn after bomb hit.

There are two layers of detection:

- Unity callbacks: `OnTriggerEnter` and `OnCollisionEnter`.
- Manual overlap fallback: `TryHandlePickupsAndHazardsByOverlap`.

The overlap fallback is useful because the bot movement currently moves the transform directly. Direct transform movement can sometimes miss physics trigger events, especially with fast objects or odd collider setup. The fallback checks bot bounds against coin/bomb bounds each frame and handles the pickup/hazard if they intersect.

Coin behavior:

- The bot records the coin so it cannot be collected twice.
- Reward is added.
- The coin object is destroyed.

Bomb behavior:

- The bot records the bomb so the same bomb cannot hit twice.
- Bomb penalty is added.
- The bot teleports to the last safe checkpoint.
- Speed resets.
- The bomb object is destroyed.

4.8 BotCheckpointTrigger.cs

This is an optional helper trigger for bot checkpoint order.

If the scene uses these objects, each one should have a `checkpointIndex`. The bot then completes checkpoint 0, then 1, and so on. This avoids relying on hierarchy order.

4.9 BotRewardSystem.cs

This stores reward values and total reward.

It is used by:

- `BotAgentRewards` to add progress, checkpoint, coin, bomb, and time rewards.
- `RLBotBrain` to learn from reward changes.
- `BotDebugUI` to display reward on screen.

This is not a full training system yet. It is enough for Stage 3 to show reward feedback and make the RL brain not be empty.

4.10 FSMBotBrain.cs

This is the main bot brain for Stage 3.

The FSM brain does not move the ship itself. It calculates input and sends it to `Bot.SetAiInput`.

Each tick:

- Ask `BotAgent` for the current target position.
- Convert target direction into local bot space.
- Turn yaw toward local X.
- Turn pitch toward local Y.
- Pick a desired speed.
- Accelerate or brake toward that speed.

Speed rules:

- Go faster when aligned.
- Slow down for turns.
- Slow down when near checkpoints.
- Slow down when avoiding bombs.
- Slow down heavily during recovery.
- If the target is behind the bot, keep turning in the last useful yaw direction.

4.11 RLBotBrain.cs

This is the basic reinforcement learning implementation.

It uses a small Q-table:

- State comes from `BotState` plus rough target direction.
- Actions are steering biases: neutral, left, right, down, up.
- It chooses randomly sometimes using `explorationRate`.
- Otherwise it chooses the highest-valued action.
- It updates the previous action value from reward delta.

Important limitation: this is not a serious trained racing agent yet. It still uses target steering as its base behavior, then adds small action bias. That is good enough for Stage 3 because the assignment asks for basic RL, not final training.

4.12 `BotManager.cs`

`BotManager` owns the scene-level bot controls.

Main jobs:

- Spawn bots from `botPrefab`.
- Remove one bot or all bots.
- Cap the number of bots using `MAX_BOTS`.
- Name bots as they spawn.
- Set bot spawn position/rotation.
- Reset bots to the spawn point.
- Cycle camera view through player and bots.
- Toggle AI gizmos.

`SwitchViewToBot` cycles like this:

- player view,
- bot 1,
- bot 2,
- more bots,
- back to player view.

`ToggleBotAIGizmos` changes the static `BotAgent.showAiGizmos` flag, so all bot agents draw or hide their debug gizmos together.

4.13 `BotDebugUI.cs`

This displays live information about one tracked bot.

It shows:

- bot count,
- tracked bot index,
- AI type,
- current state,

- current checkpoint index,
- passed checkpoints,
- current target,
- nearest checkpoint,
- coin target,
- bomb target,
- speed,
- reward.

It updates on unscaled time, so it can still refresh while the game is paused.

5 Non-AI Scripts Used By Bots

5.1 Checkpoint.cs

For the player, a checkpoint records that the player's root transform passed through it, updates the last checkpoint, and increments `RaceManager.passedCheckpoints`.

It does not destroy or disable the whole checkpoint. Instead, it only hides `portalSurface` for the player-visible effect. This matters because bots still need the trigger and bounds alive after the player passes through.

For bots, `Checkpoint.OnTriggerEnter` forwards to `BotAgent.TryCompleteCheckpointIndex`.

5.2 FinishTrigger.cs

The finish line supports both player and bots.

- For a bot, it checks `BotAgent.HasPassedAllCheckpoints`.
- If true, it calls `RaceManager.FinishRace`.
- If false, it logs that the bot reached finish early.
- For the player, it calls `RaceManager.TryFinish`.

5.3 RaceManager.cs

`RaceManager` owns player race state: pause, timer, score, player checkpoints, respawn, finish, and scene restart.

The bot integration points are:

- `botManager` reference.
- `SetLastCheckpoint` updates bot spawn point too.
- `FinishRace` lets the finish line end the race for either player or bot.

5.4 PauseMenu.cs

This file stays thin. It exposes methods Unity buttons can call:

- `AddBot`
- `AddAllBots`
- `RemoveBot`
- `RemoveAllBots`
- `SwitchViewToBot`
- `ToggleBotAIGizmos`

Actual bot logic stays in `BotManager`.

5.5 `FollowCamera.cs`

`FollowCamera` normally follows the player. Bot camera support adds:

- `Target` getter.
- `SetTarget`.
- `SnapToTargetNow`.

The offset is preserved, so the camera can follow a bot using the same chase-camera style as the player.

5.6 `Coin.cs`

Player coin logic:

- rotates the coin visually,
- checks for player trigger,
- adds score,
- destroys the coin.

Bots do not use this script directly for scoring. Bot coin pickup is handled inside `BotAgentCollision` because bot reward is separate from player score.

5.7 `Bomb.cs`

Player bomb logic:

- floats the bomb up and down,
- checks for player trigger,
- respawns the player at last checkpoint,
- destroys the bomb.

Bot bomb behavior is separate in `BotAgentCollision`: the bot gets a reward penalty, teleports to its last safe checkpoint, resets speed, and destroys the bomb.

6 How Checkpoints Work For Bots

The bot has its own checkpoint fields:

- `currentCheckpointIndex`: what the bot should hit next.
- `passedCheckpointCount`: how many checkpoints the bot completed.
- `lastSafeCheckpointIndex`: where it should respawn after a bomb.

The bot only counts a checkpoint when:

- it is the expected checkpoint index,
- its bounds overlap the checkpoint pass bounds or a trigger fires,
- it is moving through the checkpoint in the expected direction.

This prevents the old problem where the bot got close and switched target without actually passing through. The target changes only after completion is accepted.

7 How Coins And Bombs Work For Bots

Bots do not rely only on Unity trigger callbacks for coins and bombs. Because the bot moves by changing transform position directly, trigger callbacks can be unreliable. So `BotAgentCollision` also runs an overlap scan.

Coin flow:

1. Bot bounds intersect coin bounds, or a trigger/collision event fires.
2. `HandleCoinPickup` calls `CollectCoin`.
3. Reward is added.
4. Coin is destroyed.

Bomb flow:

1. Bot bounds intersect bomb bounds, or a trigger/collision event fires.
2. `HandleBombHit` adds bomb penalty.
3. Bot teleports to last safe checkpoint.
4. Bot speed resets to minimum.
5. Bomb is destroyed.

8 How Recovery Works

Recovery exists because a spaceship cannot instantly turn like a ground car. If the bot overshoots a checkpoint and keeps trying to turn directly into it, it can orbit forever.

The current recovery logic does this instead:

1. Detect that the bot is too far, too badly angled, or has passed the checkpoint plane.
2. Pick a setup point before the checkpoint, based on checkpoint pass direction.
3. Fly to that setup point at slower speed.
4. Once it is far enough and facing well enough, exit recovery.
5. Aim through the checkpoint again.

This makes the bot go away for a bit, turn around, and re-enter properly instead of circling the portal forever.

9 Debugging And Gizmos

`BotAgent.OnDrawGizmos` draws:

- green line/sphere for current target,
- cyan line/sphere for recovery setup point,
- yellow line/sphere for coin target,
- red line/sphere for bomb danger.

These are Unity Gizmos. They normally show in the Scene view. In Game view they only show if the Game view Gizmos toggle is enabled in the Unity editor. They are not normal runtime UI.

For actual always-visible game UI, use `BotDebugUI`.

10 Tuning Notes

Useful values to tune in the Inspector:

- `Bot.maxSpeed`: hard max ship speed.
- `Bot.acceleration`: how quickly the bot speeds up.
- `Bot.brakingAcceleration`: how quickly it slows down.
- `Bot.yawSpeed`: how fast it can turn horizontally.
- `FSMBotBrain.cruiseSpeed`: normal race speed.
- `FSMBotBrain.checkpointSpeed`: speed near checkpoints.
- `FSMBotBrain.recoverySpeed`: speed while lining up again.
- `BotAgent.bombAvoidDistance`: how close bombs must be before avoidance changes target.
- `BotAgent.bombAvoidSideRange`: sideways bomb corridor.
- `BotAgent.bombAvoidVerticalRange`: vertical bomb corridor.
- `BotAgent.coinInfluence`: how much coins pull the target away from checkpoint line.
- `BotAgent.recoverySetupDistance`: how far back the bot goes to line up again.

If the bot misses checkpoints, start by lowering checkpoint speed and recovery speed. If it dodges checkpoints near bombs, reduce bomb avoidance distance or side/vertical range. If it ignores coins, increase coin detection radius or coin influence.

11 What Is Still Basic

This system is good for Stage 3, but not final AI.

Things that are still intentionally simple:

- No real path planning around level geometry.
- No trained RL model saved to disk.
- RL values reset when the scene starts.
- Coins are a steering bias, not a planned route.
- Bomb avoidance is corridor-based, not a full avoidance solver.
- Movement uses direct transform updates, so overlap fallback is needed for reliable pickups.

For Stage 4, the natural next step is to keep FSM as the reliable baseline and make RL compare against it with better observations, stored training data, and more controlled reward shaping.